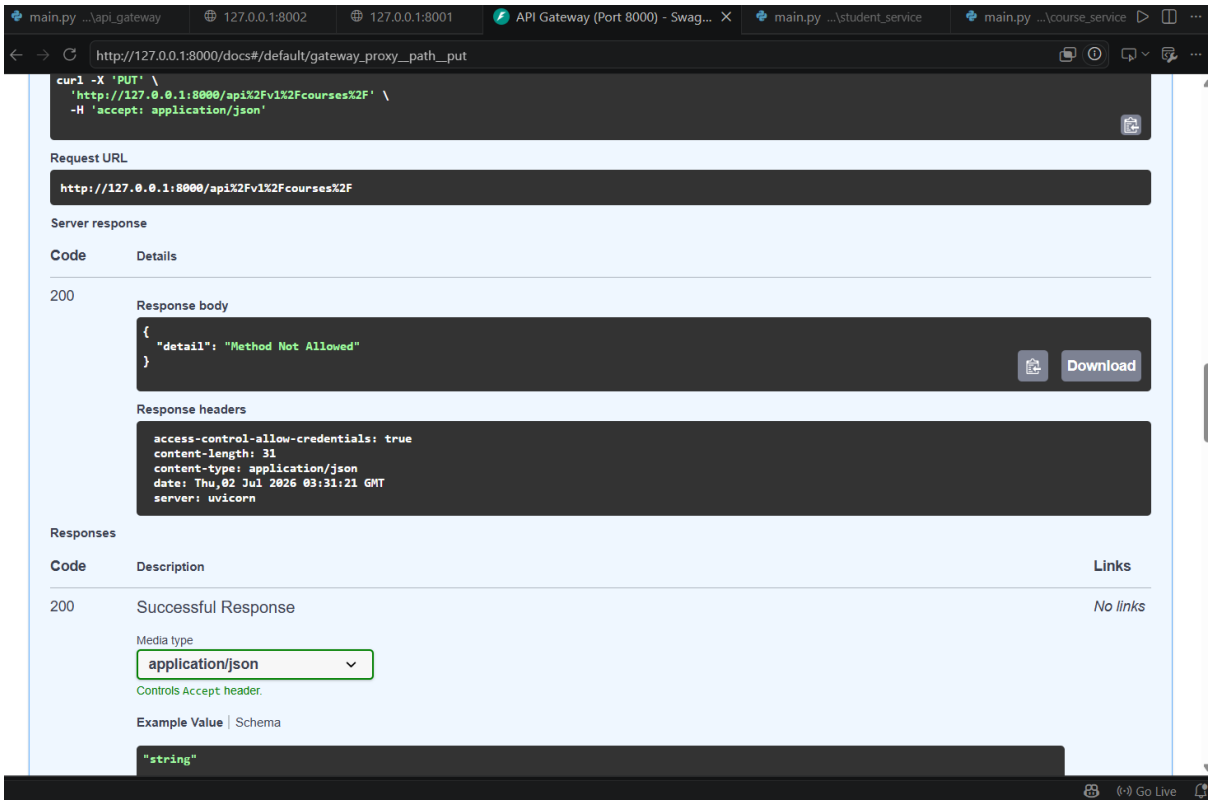
HANDS-ON 9 [Advanced]
Authentication & Security — JWT, OAuth2 & OWASP
Topics Covered:
JWT Token Structure✓
Token-Based Auth vs Session-Based Auth✓
Password Hashing with bcrypt✓
OAuth2 Flow (concept)✓
CORS Configuration✓
OWASP Top 10 Awareness✓
Security is non-negotiable in any real API. In this hands-on, you will implement JWT-based authentication, protect routes, hash passwords, configure CORS, and review the most common web vulnerabilities from the OWASP Top 10.
INSTALL
pip install python-jose[cryptography] passlib[bcrypt] python-multipart
Task 1: Password Hashing and User Registration
Goal: Implement secure password storage using bcrypt hashing.
Steps:
86.
Create a User model with fields: id, email (unique), hashed_password, is_active.
87.
In a security.py utility file, implement: get_password_hash(password: str) -> str using passlib's CryptContext with bcrypt scheme, and verify_password(plain_password, hashed_password) -> bool.
88.
Create a POST /api/v1/auth/register/ endpoint that: validates the email format, checks the email is not already registered (return 409 Conflict if so), hashes the password using get_password_hash, and saves the user.
89.
Never store or log the plain-text password at any point. Add a comment in the code explaining why bcrypt is preferred over MD5 or SHA-256 for passwords.
90.
Test: register a user, then inspect the database — confirm only the hashed password is stored, never the plain text.
Hint:
•
bcrypt intentionally slow hashing (work factor) makes brute-force attacks computationally expensive — unlike MD5/SHA which are designed to be fast.
•
Always check if the email is already registered before inserting — a UNIQUE constraint on the DB prevents duplicates, but returning a 409 gives the client a meaningful error.
Expected Outcome: Registration stores only the bcrypt hash. A second registration with the same email returns 409 Conflict.
Task 2: JWT Login, Protected Routes and CORS

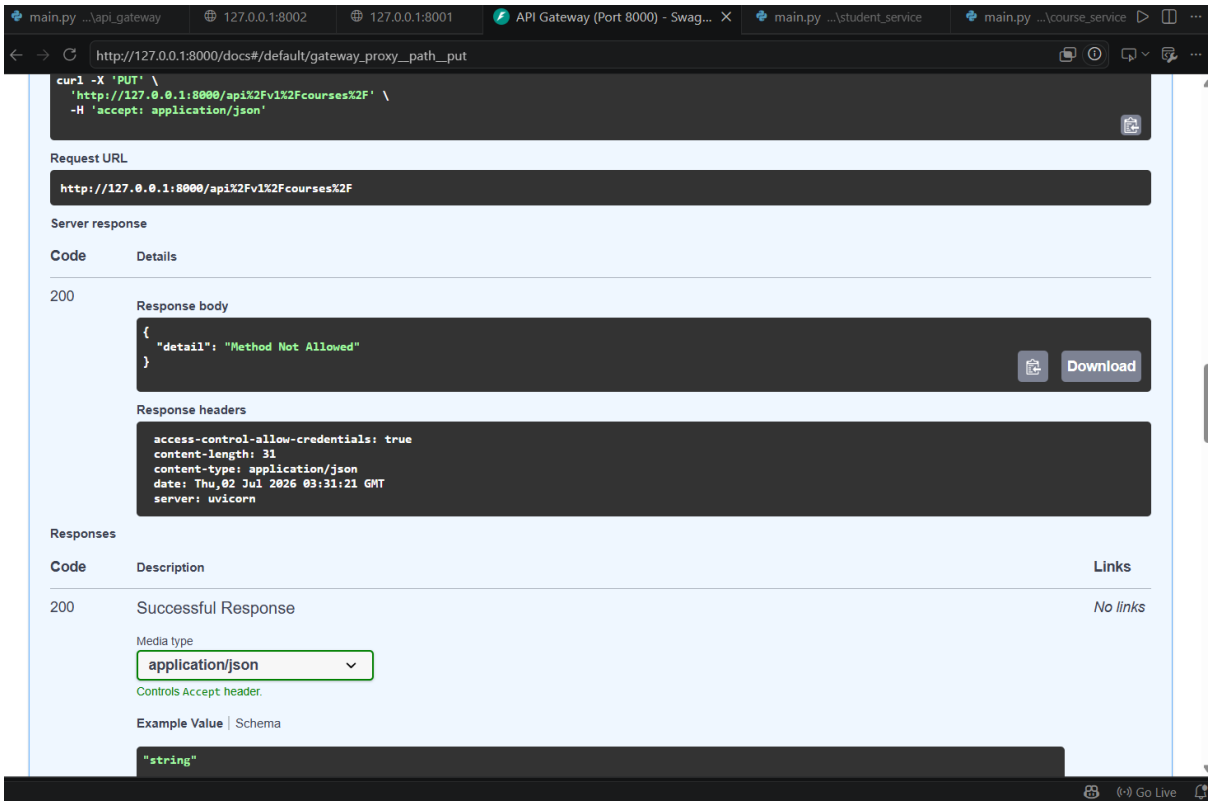
Goal: Implement JWT token generation, route protection, and CORS configuration.
Steps:
91.
Create a POST /api/v1/auth/login/ endpoint that: accepts email and password, verifies credentials using verify_password, creates a JWT token using python-jose with an expiry of 30 minutes, and returns {'access_token': token, 'token_type': 'bearer'}.
92.
Create a get_current_user(token: str = Depends(oauth2_scheme)) dependency that decodes and validates the JWT token, returning the current user object. Raise 401 if the token is invalid or expired.
93.
Protect the POST /api/v1/courses/ and DELETE /api/v1/courses/{id}/ endpoints by adding current_user: User = Depends(get_current_user) as a parameter. Unauthenticated requests should receive 401.
94.
Configure CORS in your app to allow requests from http://localhost:3000 (your frontend dev server). In FastAPI: use CORSMiddleware. In Django: use django-cors-headers. In Flask: use flask-cors.

95.
Document in comments: what is the OAuth2 Authorization Code flow, and how does it differ from the simple JWT login you just implemented?
Hint:
•
JWT payloads are base64-encoded, not encrypted — never store sensitive data (passwords, credit cards) in a JWT payload.
•
CORS headers are sent by the server to tell the browser which origins are allowed. CORS is enforced by the browser, not the server — server-side APIs are not protected by CORS.
Expected Outcome: Login returns a valid JWT. GET /api/v1/courses/ works without auth. POST /api/v1/courses/ returns 401 without a valid token. CORS allows localhost:3000.

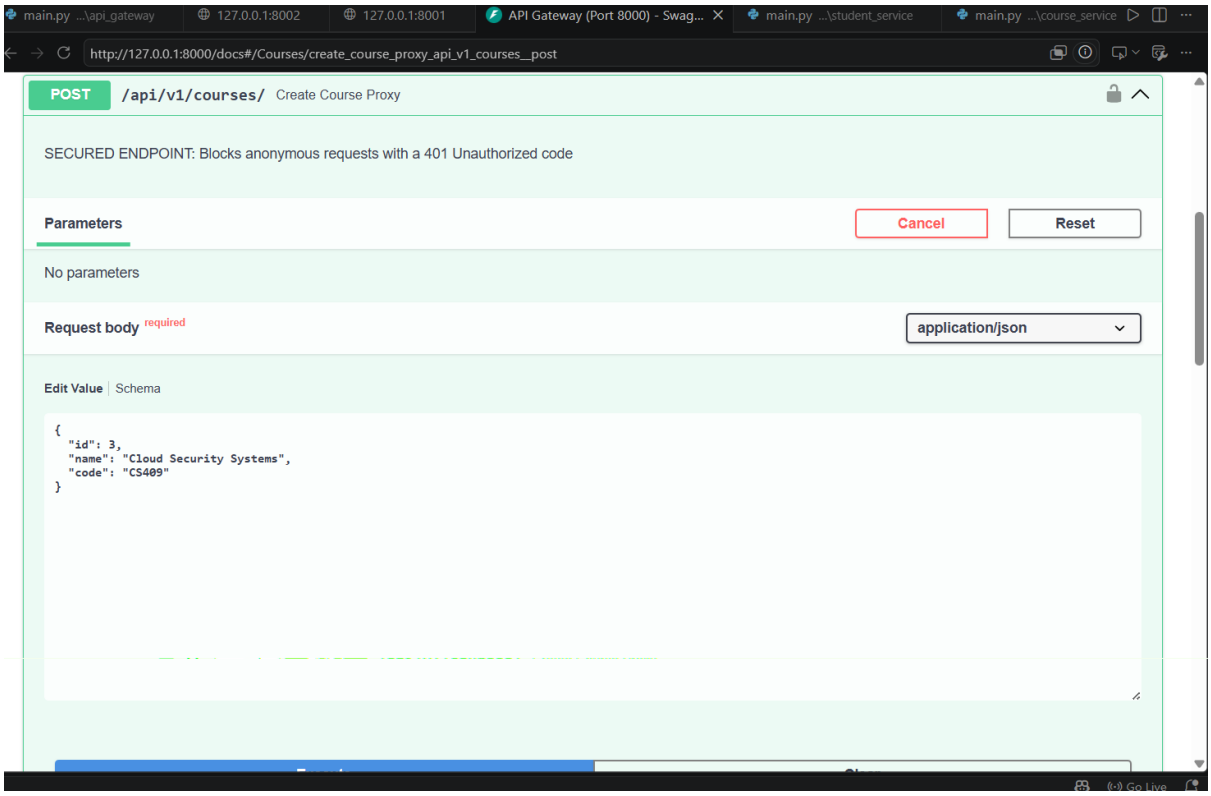
OUTCOME:

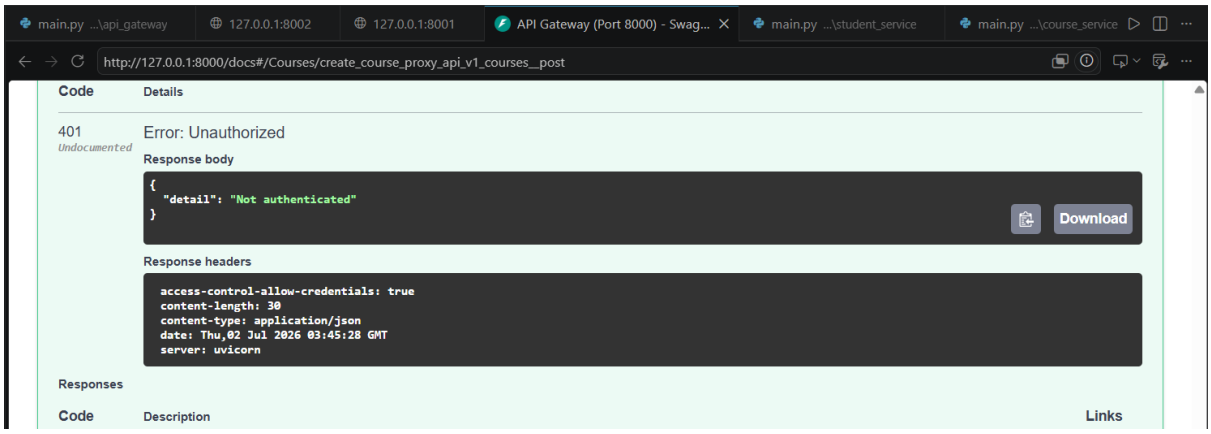
GET:


main.py ...\api_gateway
127.0.0.1:8002
127.0.0.1:8001
API Gateway (Port 8000) - Swag...
main.py ...\student_service
main.py ...\course_service
http://127.0.0.1:8000/docs#/default/gateway_proxy__path__put
curl -X 'PUT' \
'http://127.0.0.1:8000/api%2Fv1%2Fcourses%2F' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/api%2Fv1%2Fcourses%2F
Server response
Code
Details
200
Response body
{
"detail": "Method Not Allowed"
}
Download
Response headers
access-control-allow-credentials: true
content-length: 31
content-type: application/json
date: Thu,02 Jul 2026 03:31:21 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
"string"
Go Live


main.py ...\api_gateway
127.0.0.1:8002
127.0.0.1:8001
API Gateway (Port 8000) - Swag...
main.py ...\student_service
main.py ...\course_service
http://127.0.0.1:8000/docs#/default/gateway_proxy__path__put
curl -X 'PUT' \
'http://127.0.0.1:8000/api%2Fv1%2Fcourses%2F' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/api%2Fv1%2Fcourses%2F
Server response
Code
Details
200
Response body
{
"detail": "Method Not Allowed"
}
Download
Response headers
access-control-allow-credentials: true
content-length: 31
content-type: application/json
date: Thu,02 Jul 2026 03:31:21 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
"string"
Go Live

POST:


main.py ...\api_gateway
127.0.0.1:8002
127.0.0.1:8001
API Gateway (Port 8000) - Swag...
main.py ...\student_service
main.py ...\course_service
http://127.0.0.1:8000/docs#/Courses/create_course_proxy_api_v1_courses__post
POST
/api/v1/courses/
Create Course Proxy
SECURED ENDPOINT: Blocks anonymous requests with a 401 Unauthorized code
Parameters
Cancel
Reset
No parameters
Request body
required
application/json
Edit Value
Schema
{
"id": 3,
"name": "Cloud Security Systems",
"code": "CS409"
}


main.py ...\api_gateway
127.0.0.1:8002
127.0.0.1:8001
API Gateway (Port 8000) - Swag...
main.py ...\student_service
main.py ...\course_service
http://127.0.0.1:8000/docs#/Courses/create_course_proxy_api_v1_courses__post
Code
Details
401
Undocumented
Error: Unauthorized
Response body
{
"detail": "Not authenticated"
}
Download
Response headers
access-control-allow-credentials: true
content-length: 30
content-type: application/json
date: Thu,02 Jul 2026 03:45:28 GMT
server: uvicorn
Responses
Code
Description
Links